Transformers and ACT

From observations to a sequence of actions

Action Chunking with Transformers (ACT) predicts several robot actions together.

How can a robot combine several observations and predict a coordinated movement?



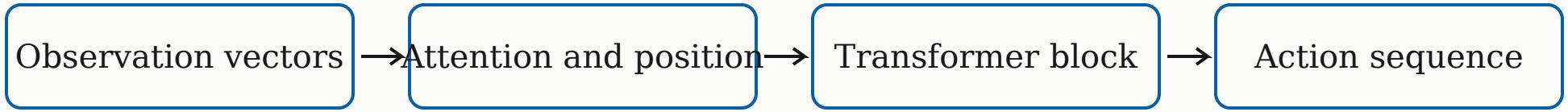
Zhao et al. (2023), §IV-B–C, Appendix C

ROADMAP

Transformers and ACT

The mathematical route

Attention selects information. Position identifies order. A policy turns the resulting representation into actions.



Vaswani et al. (2017), §3.2.1, Eq. 1

DEFINITION

Tokens and projections

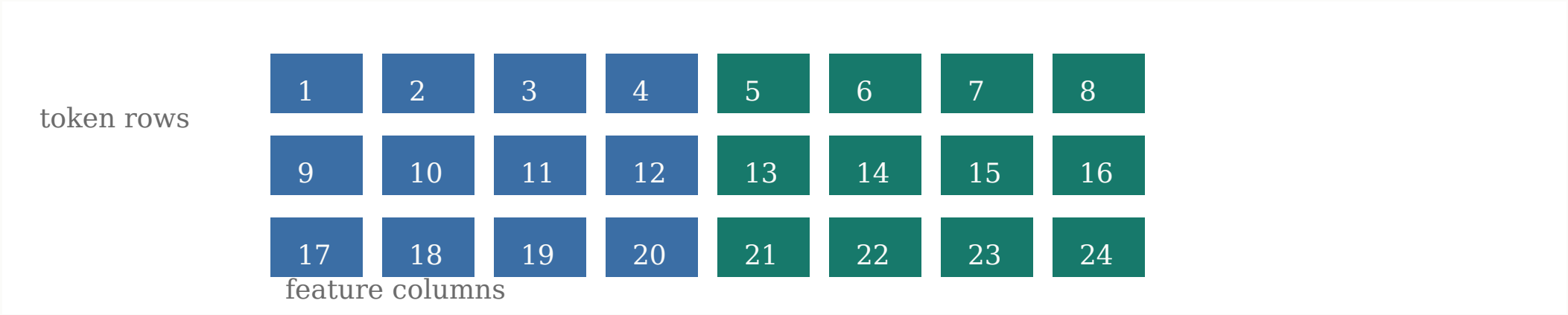
A token is a feature vector

A token represents one item, such as an image patch, with a feature vector.

$$X = [x_1 \ \cdots \ x_T]^T \in \mathbb{R}^{T \times D}$$

X, x_i token matrix; feature column for token i

T, D token count; features per token



Rows identify tokens; columns identify features.

Vaswani et al. (2017), §3.2.1, Eq. 1

DERIVATION

Tokens and projections

Column weights and row-stored tokens

The learned map gives each token a query vector for comparison.

$$q_i = W_Q x_i \quad \Longrightarrow \quad q_i^T = x_i^T W_Q^T \quad \Longrightarrow \quad Q = X W_Q^T$$

- x_i, q_i token and query columns of widths D and d_k
- X, Q T token rows and T query rows
- W_Q learned $d_k \times D$ map

Vaswani et al. (2017), §3.2.1, Eq. 1

DEFINITION

Tokens and projections

Three learned views of the same input

$$Q = XW_Q^T, \quad K = XW_K^T, \quad V = XW_V^T$$

X	T token rows of width D
W_Q, W_K, W_V	learned maps from D to d_k, d_k and d_v
Q, K, V	query, key and value rows of those widths

Query–key dot products set comparison scores. The scores determine how to combine the values.

Vaswani et al. (2017), §3.2.1, Eq. 1

EXAMPLE

Tokens and projections

A projection computed by hand

$$X = \begin{bmatrix} 1 & 2 & -1 \\ 0 & 1 & 3 \end{bmatrix}, \quad W_Q = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & -1 \end{bmatrix}$$

X two token rows with three features

W_Q, Q query map; matrix of query rows

$$Q = XW_Q^T = \begin{bmatrix} 0 & 5 \\ 3 & -1 \end{bmatrix}.$$

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

EXERCISE

Tokens and projections

A new token, the same learned map

Input $x = (2, -1, 1)^T$. The query map is $W_Q = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & -1 \end{bmatrix}$. Find the query $q = W_Q x$

.

A $(3, -3)^T$

B $(1, -1)^T$

C A three-coordinate vector

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

DEFINITION

Scaled dot-product attention

A score compares a query with a key

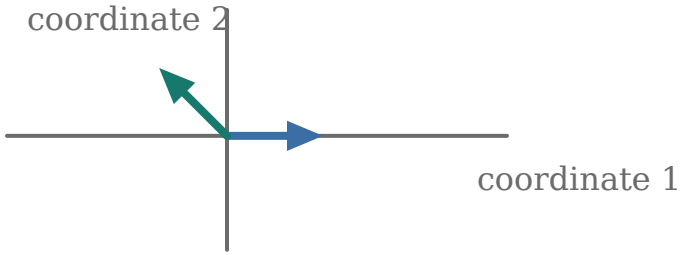
A compatibility score compares a query with a key using a scaled dot product.

$$s_{ij} = \frac{q_i^T k_j}{\sqrt{d_k}}$$

i, j query index, key index

q_i, k_j d_k-dimensional columns

s_{ij} scalar compatibility score



q fixed; key rotates

angle = 2.36 rad

q · k = -0.71

The key direction changes; the query stays fixed.

Vaswani et al. (2017), §3.2.1, Eq. 1

EXERCISE

Scaled dot-product attention

Alignment and length both affect the score

Query $q = (1, 0)^T$; keys $k_1 = (1, 0)^T$, $k_2 = (2, 0)^T$. Their width is $d_k = 2$. Which key has the larger scaled dot-product score?

A Key 1

B Key 2

C They must tie

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

DERIVATION

Scaled dot-product attention

All query-key pairs in one matrix

$$S = \frac{QK^T}{\sqrt{d_k}} \in \mathbb{R}^{T_q \times T_k}$$

Q Tq × dₖ query rows

K Tk × dₖ key rows

S_{ij} query i versus key j

Self-attention uses one token collection for queries, keys and values. Cross-attention uses one collection for queries and another for keys and values.



Each row compares one query with all keys.

Vaswani et al. (2017), §3.2.1, Eq. 1

DEFINITION

Scaled dot-product attention

Weights combine values

Softmax turns finite scores in one query row into positive weights that add to one.

$$P_{ij} = \frac{e^{S_{ij}}}{\sum_{\ell} e^{S_{i\ell}}}, \qquad y_i = \sum_j P_{ij} v_j$$

S_{ij}, P_{ij} score and weight for query i and key j

ℓ all keys in query row i

v_j, y_i d_v-dimensional value and output columns

Vaswani et al. (2017), §3.2.1, Eq. 1

EXAMPLE

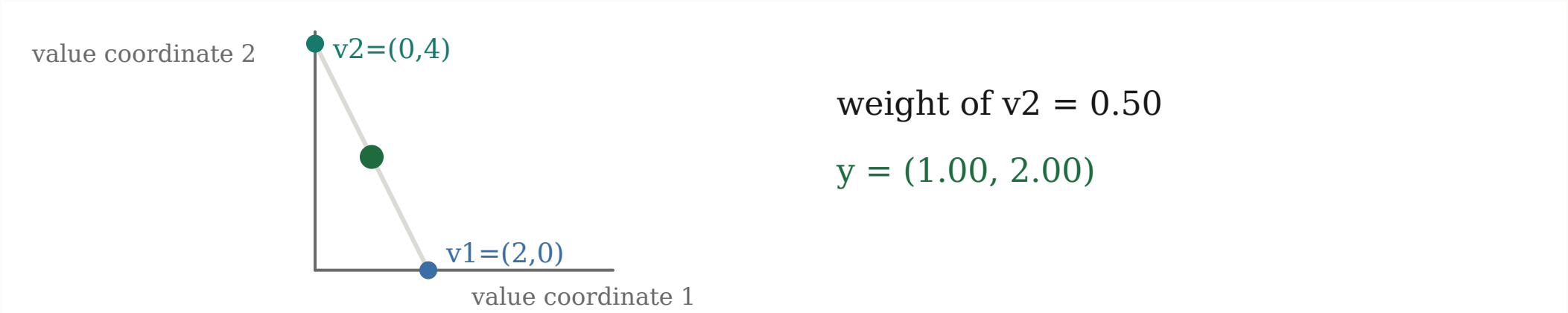
Scaled dot-product attention

Equal scores give an ordinary average

$$s = (0, 0), \quad v_1 = (2, 0)^T, \quad v_2 = (0, 4)^T$$

$s; v_1, v_2$ one query's scores; two value vectors

$p = (1/2, 1/2)$, so $y = (1, 2)^T$.



Equal weights place the output halfway between the value vectors.

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

EXERCISE

Scaled dot-product attention

Unequal scores change the mixture

Attention scores are $s = (0, \log 3)$ and values are $v_1 = (2, 0)^T$, $v_2 = (0, 4)^T$. Find the output.

A $(1, 2)^T$

B $(1/2, 3)^T$

C $(0, 4)^T$

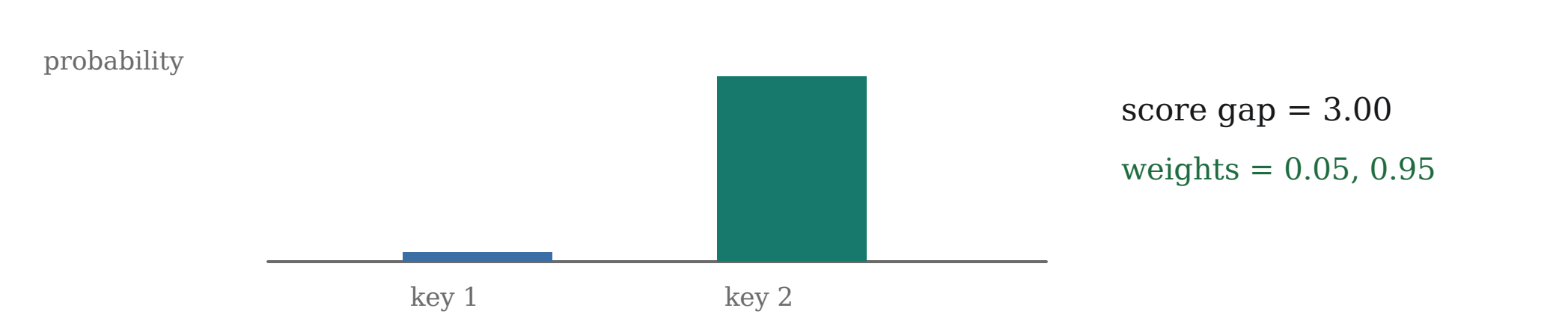
Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

MOTIVATION

Scaled dot-product attention

Why divide by the square root?

Wider query–key vectors can produce more variable dot products. A large score gap can put almost all the weight on one key.



As the score gap grows, the larger score gets more of the total weight.

Vaswani et al. (2017), §3.2.1, Eq. 1

DERIVATION

Scaled dot-product attention

Variance of one product

$$\begin{aligned}\mathbb{E}[q_r k_r] &= \mathbb{E}[q_r]\mathbb{E}[k_r] = 0 \\ \mathrm{Var}(q_r k_r) &= \mathbb{E}[q_r^2 k_r^2] - (\mathbb{E}[q_r k_r])^2 \\ &= \mathbb{E}[q_r^2]\mathbb{E}[k_r^2] = 1\end{aligned}$$

- q_r, k_r independent random coordinates, each with mean 0 and variance 1
- r one matched feature coordinate

Vaswani et al. (2017), §3.2.1, Eq. 1

DERIVATION

Scaled dot-product attention

Variance of the sum

Assume all coordinates of query q and key k are independent, with mean 0 and variance 1.

$$\mathrm{Var}(q^T k) = \sum_{r=1}^{d_k} 1 = d_k, \qquad \mathrm{Var}\left(\frac{q^T k}{\sqrt{d_k}}\right) = \frac{1}{d_k} d_k = 1$$

d_k query/key width; one coordinate product for each of the d_k coordinates

Vaswani et al. (2017), §3.2.1, Eq. 1

EXERCISE

Scaled dot-product attention

The scale is determined by the assumptions

Query and key each have $d_k = 16$ independent zero-mean, unit-variance coordinates, independent across the two vectors. What is the standard deviation of their unscaled dot product?

A 16

B 4

C 1

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

DERIVATION

Stable attention and masks

The largest logit can be subtracted

$$\frac{e^{s_j - c}}{\sum_\ell e^{s_\ell - c}} = \frac{e^{-c} e^{s_j}}{e^{-c} \sum_\ell e^{s_\ell}} = p_j$$

s_j, p_j key j score and normalized weight in one row

c shared finite shift; choose the maximum

For a finite row maximum, the largest shifted exponential is 1.

Blanchard, Higham & Higham (2021), §4

EXAMPLE

Stable attention and masks

A large-logit example

$$s = (1000, 1000 + \log 3), \quad s - \max s = (-\log 3, 0)$$

s one score row

$e^{s - \max s} = (1/3, 1)$; normalize to $(1/4, 3/4)$.

Blanchard, Higham & Higham (2021), §4 · Course example

EXERCISE

Stable attention and masks

A shift or a temperature?

A score row is $s = (0, \log 3)$. Which change preserves its softmax weights?

- **A** Subtract 5 from both entries
- **B** Multiply both entries by 2
- **C** Add 5 only to the second entry

Blanchard, Higham & Higham (2021), §4 · Course example

DEFINITION

Stable attention and masks

A causal mask forbids future keys

In causal self-attention, query and key positions share the same ordered sequence.

$$M_{ij} = \begin{cases} 0 & j \le i \\ -\infty & j > i \end{cases}, \qquad P = \mathrm{softmax}_{\mathrm{row}}(S + M)$$

i, j query and key positions

S, M, P scores, additive mask and row weights



Rows are queries; columns are keys. The upper triangle is forbidden.

Vaswani et al. (2017), §3.2.1, Eq. 1

DEFINITION

Stable attention and masks

Padding and causality answer different questions

Padding equalizes lengths with placeholders. A key-validity mask excludes them.

$$S : [B, H, T_q, T_k], \quad V_k : [B, 1, 1, T_k], \quad C : [1, 1, T_q, T_k]$$

S, V_k, C	scores, key-valid mask, causal mask
B, H	batch items, attention heads
T_q, T_k	query and key counts; True allows in either mask

Use $V_k \wedge C$. Exclude padded queries from loss.

Vaswani et al. (2017), §3.2.1, Eq. 1

EXERCISE

Stable attention and masks

Mask before normalizing

Scores are $(0, \log 3)$; the second key is forbidden. Which output weights are correct?

A $(1, 0)$

B $(1/4, 0)$

C $(1/4, 3/4)$

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

COUNTEREXAMPLE

Stable attention and masks

Every valid query needs an allowed key

An all-forbidden row has masked score $s_j = -\infty$ for every key. Softmax gives $0/0$; max shifting gives $-\infty - (-\infty)$.

Valid query: require an allowed key. Padded query: skip normalization, return zero and exclude its loss.

Vaswani et al. (2017), §3.2.1, Eq. 1

EXERCISE

Stable attention and masks

Diagnose the NaN without changing the task

A padded query has no allowed key. Stable softmax returns NaN (Not a Number). Which repair preserves the declared information rule?

A. Use uniform weights over all keys

B. Mark the query invalid and output zero

C. Remove all masks for the batch

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

EXAMPLE

Stable attention and masks

One attention query, end to end

$$q = (\sqrt{2}, 0)^T, \quad K = \begin{bmatrix} 0 & 1 \\ \log 3 & 0 \end{bmatrix}, \quad V = \begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix}$$

q, K, V	query column; key rows; value rows
k_1, k_2	key columns formed from rows of K
$d_k = 2$	features per query/key; both keys allowed

Dot products $(0, \sqrt{2}\log 3)$ give scaled scores $s = (0, \log 3)$.

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

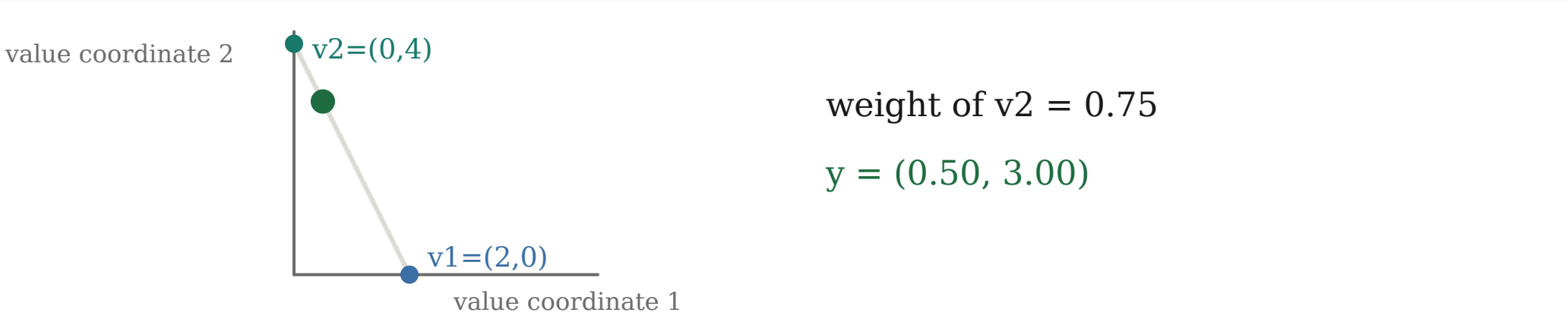
EXAMPLE

Stable attention and masks

The output follows the same data path

$$s = (0, \log 3) \Rightarrow p = (1/4, 3/4) \Rightarrow y = (1/2, 3)^T$$

- s scaled scores
- p row weights
- y mixture of $(2,0)^T$ and $(0,4)^T$



Weights (1/4, 3/4) place the output at (1/2, 3).

Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

EXERCISE

Stable attention and masks

Transfer: change the permitted information

Query $q = (\sqrt{2}, 0)^T$; keys $k_1 = (0, 1)^T$, $k_2 = (\log 3, 0)^T$; values $v_1 = (2, 0)^T$, $v_2 = (0, 4)^T$. Feature width $d_k = 2$. Only key 1 is allowed. Find the output.

A $(1/2, 3)^T$

B $(2, 0)^T$

C $(0, 0)^T$

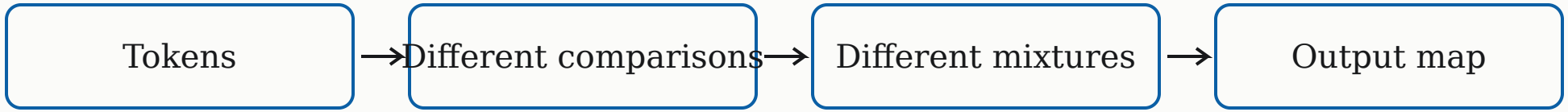
Vaswani et al. (2017), §3.2.1, Eq. 1 · Course example

MOTIVATION

Multi-head attention

One mixture can hide useful differences

An attention head computes one learned set of comparisons and value mixtures. Multi-head attention (MHA) runs several heads, concatenates their outputs, and projects the result.



Vaswani et al. (2017), §3.2.2

DEFINITION

Multi-head attention

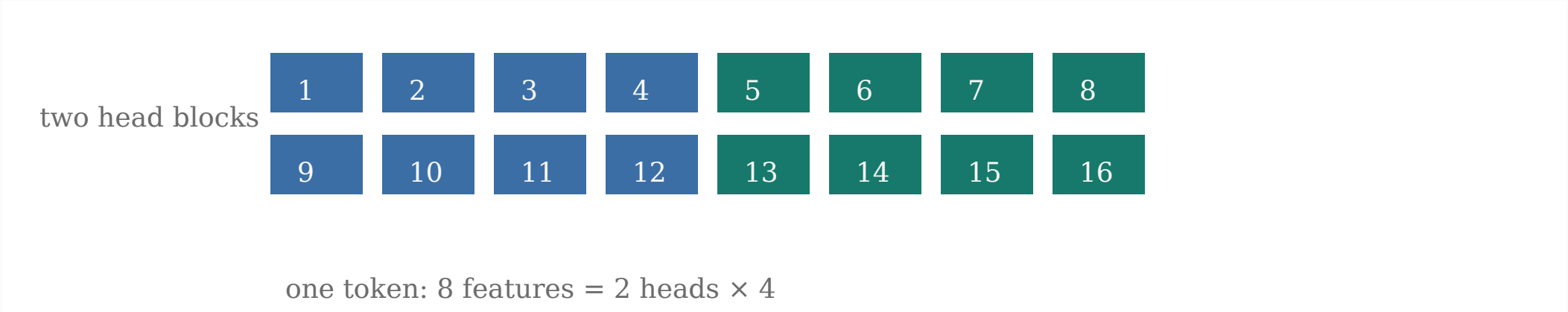
A feature axis can be split into heads

$$Q : [B, T, D] \to [B, T, H, d_h], \qquad D = H d_h$$

Q, X, W_Q query tensor; token rows; query map (Wq in code)

B, T, H, d_h, D batch, tokens, heads, features per head, packed width

```
Q = (X @ Wq.T).reshape(B,T,H,dh)
```



A packed feature row splits into two head blocks.

Vaswani et al. (2017), §3.2.2

DERIVATION

Multi-head attention

Transpose puts matrix axes last

$$Q, K : [B, T, H, d_h] \longrightarrow [B, H, T, d_h], \qquad QK^{\mathsf{T}_{\text{last}}} : [B, H, T, T]$$

Q, K	query and key tensors
B, H	batch items and heads
T, d_h	tokens and features per head

Transpose the last two axes of K only for the product. The score axes end in query and key.

Vaswani et al. (2017), §3.2.2

EXERCISE

Multi-head attention

Trace a tensor through the heads

Self-attention starts with queries Q in batch, token, feature order. Batch $B = 2$, tokens $T = 3$, width $D = 8$, heads $H = 2$. Find query and score (S) shapes after splitting and transposing.

A Q [2,2,3,4]; S [2,2,3,3]

B Q [2,3,2,4]; S [2,3,2,2]

C Q [2,2,3,8]; S [2,2,8,8]

Vaswani et al. (2017), §3.2.2 · Course example

DEFINITION

Multi-head attention

Concatenate, then mix across heads

$$C = \mathrm{Concat}(Y^{(1)}, \ldots, Y^{(H)}), \quad Y = CW_O^T$$

$Y^{(h)}$	head h output for one item, T tokens × d_v value features
H, D	head count; output width
C, Y	concatenated and projected outputs
W_O	D × Hd_v learned output map

```
C = heads.transpose(1,2).reshape(B,T,H*dv)
out = C @ Wo.T
```

Vaswani et al. (2017), §3.2.2

EXAMPLE

Multi-head attention

Two heads, one explicit output map

$$c = (1, 2, 3, -1)^T, \quad W_O = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

c head 1: (1,2), head 2: (3,−1)

W_O learned output map

$W_O c = (1 + 3, 2 - 1)^T = (4, 1)^T.$

Vaswani et al. (2017), §3.2.2 · Course example

EXERCISE

Multi-head attention

Transfer: a different output map

Concatenated heads $c = (1, 2, 3, -1)^T$. The output map is $W_O = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & -1 \end{bmatrix}$.

Find $W_O c$.

A $(3, 2)^T$

B $(4, 1)^T$

C $(1, 2, 3, -1)^T$

Vaswani et al. (2017), §3.2.2 · Course example

IMPLEMENTATION

Multi-head attention

The forward pass follows the equations

```
Q,K,V: [B,H,T,dh]
S = Q @ K.transpose(-2,-1) / sqrt(dh)
P = masked_softmax(S, allowed, query_valid)
head_outputs = P @ V
```

Query/key/value axes: batch, head, token, feature. S: scores. P: weights.

A valid query needs an allowed key. Normalize over those keys. Padded queries get zero weights.

Vaswani et al. (2017), §3.2.2

EXERCISE

Multi-head attention

Correct shapes can hide a wrong normalization

Score tensor S has axes [B,H,Tq,Tk]: batch, head, query, key. A programmer uses `softmax(S,dim=-2)`. What is normalized?

A. Keys for each query

B. Queries for each key

C. Features of each token

Vaswani et al. (2017), §3.2.2 · Course example

EXERCISE

Multi-head attention

Transfer: broadcast a key-validity mask

Scores have shape [2,3,4,5]: batch, heads, queries, keys. Key validity has shape [2,5]. Which reshape shares the key rule across every head and query?

A [2,1,1,5]

B [2,5,1,1]

C [1,1,2,5]

Vaswani et al. (2017), §3.2.2 · Course example

DERIVATION

Position and order

Content-only attention follows a permutation

Permutation equivariance means that reordering input tokens reorders their output vectors in the same way.

$$\mathrm{Attn}(\Pi X) = \Pi\,\mathrm{Attn}(X)$$

- Π, X token permutation; token rows
- Attn attention with no position terms or fixed mask

Vaswani et al. (2017), §3.5

EXERCISE

Position and order

What does content alone fail to identify?

The same two feature vectors are presented in opposite orders to unmasked attention with no position features.

A. Dot products recover their original order

B. Outputs move with the input permutation

C. Every output becomes zero

Vaswani et al. (2017), §3.5 · Course example

DEFINITION

Position and order

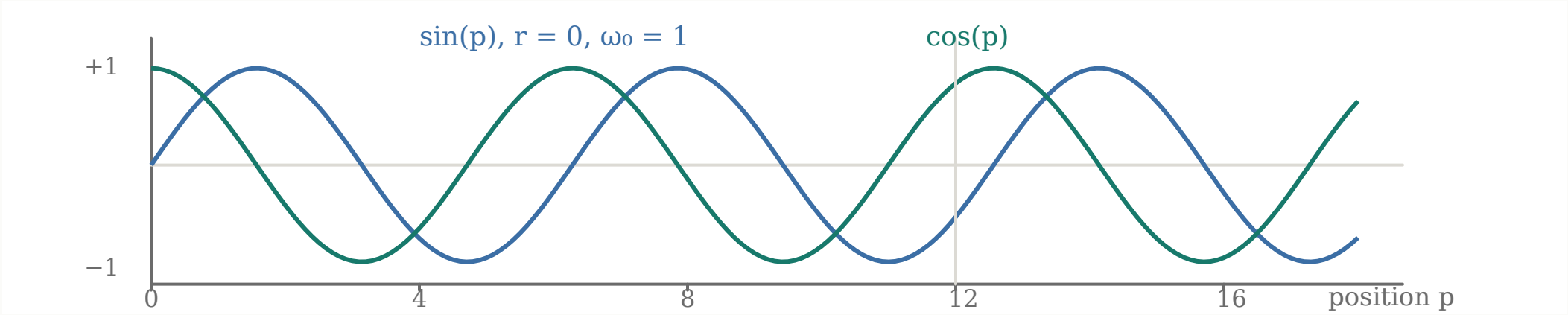
Sinusoids give each position a pattern

A positional encoding is a vector that identifies a token position. Add it to the token features before projection.

$$\mathrm{PE}_{p,2r} = \sin(p\omega_r), \quad \mathrm{PE}_{p,2r+1} = \cos(p\omega_r)$$

p token position

$\omega_r = 10000^{-2r/D}$ frequency; even width D, r=0,…,D/2−1



One sine/cosine pair across token positions.

Vaswani et al. (2017), §3.5

DERIVATION

Position and order

A shift follows angle-addition identities

$$\begin{bmatrix}\sin((p+\Delta)\omega)\\ \cos((p+\Delta)\omega)\end{bmatrix} = \begin{bmatrix}\cos(\Delta\omega) & \sin(\Delta\omega)\\ -\sin(\Delta\omega) & \cos(\Delta\omega)\end{bmatrix}\begin{bmatrix}\sin(p\omega)\\ \cos(p\omega)\end{bmatrix}$$

p, Δ, ω position, offset, one frequency

Vaswani et al. (2017), §3.5

EXERCISE

Position and order

Transfer: one sinusoidal pair at a new position

One positional pair is $(\sin(p\omega), \cos(p\omega))$. The angle per position is $\omega = \pi/2$. Move from position $p = 0$ to $p = 1$. What is the new pair?

A $(1,0)$

B $(0,1)$

C $(-1,0)$

Vaswani et al. (2017), §3.5 · Course example

DEFINITION

Position and order

Rotary embeddings act inside the comparison

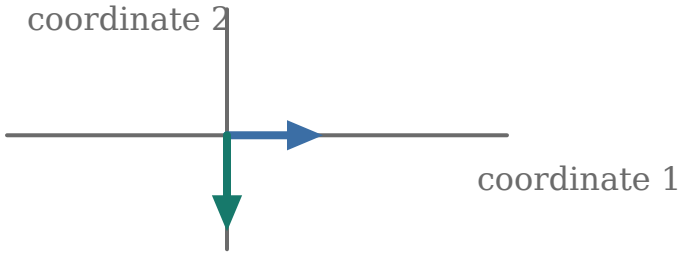
Rotary position embedding (RoPE) rotates feature pairs by position.

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

θ, ω rotation angle; angle per position

q_p, k_s query/key at positions p and s

Query/key rotation angles: $p\omega, s\omega$.



unit length after rotation

angle = 4.71 rad

A query pair rotates with position while keeping its length.

Su et al., RoFormer (2021), §3.2

DERIVATION

Position and order

Relative positions appear in the dot product

$$\tilde{q}_p^T \tilde{k}_s = q_p^T R(p\omega)^T R(s\omega) k_s = q_p^T R((s-p)\omega) k_s$$

q_p, k_s	original query/key pairs at positions p and s
$\tilde{q}_p, \tilde{k}_s$	rotated query/key pairs
$R(\theta), \omega$	two-coordinate rotation; angle per position

Su et al., RoFormer (2021), §3.2

EXAMPLE

Position and order

A quarter-turn example

$$q = k = (1, 0)^T, \quad p = 0, \quad s = 1, \quad \omega = \pi/2$$

q, k	original query and key
p, s	their positions
ω	angle per position

After position rotations, $\tilde{q} = (1, 0)^T$ and $\widetilde{k} = (0, 1)^T$. Their dot product is 0.

Su et al., RoFormer (2021), §3.2 · Course example

EXERCISE

Position and order

Transfer: shift both positions

Query and key are $q = k = (1, 0)^T$. A tilde rotates each vector counterclockwise by its position times $\omega = \pi/2$. Move (p, s) from $(0, 1)$ to $(2, 3)$. Find $\tilde{q}_p^T \tilde{k}_s$.

A 0

B 1

C -1

Su et al., RoFormer (2021), §3.2 · Course example

DEFINITION

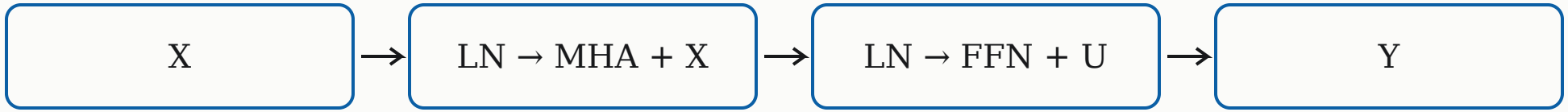
Transformer blocks

The pre-LN block has two residual paths

Residual connections add branch outputs to inputs. Pre-LN normalizes before each branch.

$$U = X + \mathrm{MHA}(\mathrm{LN}(X)), \quad Y = U + \mathrm{FFN}(\mathrm{LN}(U))$$

X, U, Y	T × D representations
MHA, FFN	multi-head attention; feed-forward network
LN	layer normalization within each token



Xiong et al. (2020), §3.1, Table 1

RECALL

Transformer blocks

LayerNorm uses the token feature axis

$$\widehat{x}_r = \frac{x_r - \mu}{\sqrt{v + \epsilon}}, \quad \mu = \frac{1}{D}\sum_r x_r, \quad v = \frac{1}{D}\sum_r (x_r - \mu)^2$$

x_r, D	feature r of one token; feature count
$\mu, v, \epsilon > 0$	feature mean, variance, stability constant
$\widehat{x}$	normalized features

Output: $\gamma \odot \widehat{x} + \beta$. Learned scale γ acts elementwise; β is the learned shift.

Xiong et al. (2020), §3.1, Table 1

EXERCISE

Transformer blocks

Transfer: normalize one token

Apply LayerNorm to the features of one token $x = (1, 3)$. Use scale $\gamma = (1, 1)$, shift $\beta = (0, 0)$ and stability constant $\epsilon = 0$ for this nonconstant example. Find the output.

A $(-1,1)$

B $(-1/\sqrt{2},\ 1/\sqrt{2})$

C $(1,3)$

Xiong et al. (2020), §3.1, Table 1 · Course example

DEFINITION

Transformer blocks

The feed-forward branch transforms each token

A positionwise feed-forward network applies the same nonlinear feature map independently to every token.

$$f(x) = W_2\phi(W_1x + b_1) + b_2$$

x, D, F	token features, input width, hidden width
W_1, W_2	F × D and D × F maps
ϕ, b_1, b_2	elementwise nonlinearity; F- and D-coordinate biases

Vaswani et al. (2017), §3.3

EXERCISE

Transformer blocks

A residual connection constrains the shape

X has shape [2,5,12], while the attention branch returns [2,5,8]. Can they be directly added?

A. Yes, ordinary broadcasting repairs it

B. No, the branch must return width 12

C. Yes, because both have five tokens

Xiong et al. (2020), §3.1, Table 1 · Course example

EXERCISE

Transformer blocks

Transfer: a missing path changes the architecture

A proposed block is $Y = X + \mathrm{FFN}(\mathrm{MHA}(X))$. X, Y are input/output representations, MHA is multi-head attention, and FFN is a feed-forward network. What is missing for pre-layer normalization (pre-LN)?

A Only position information

B Both normalizations and the intermediate residual

C A final softmax only

Xiong et al. (2020), §3.1, Table 1 · Course example

DERIVATION

Cost and the KV cache

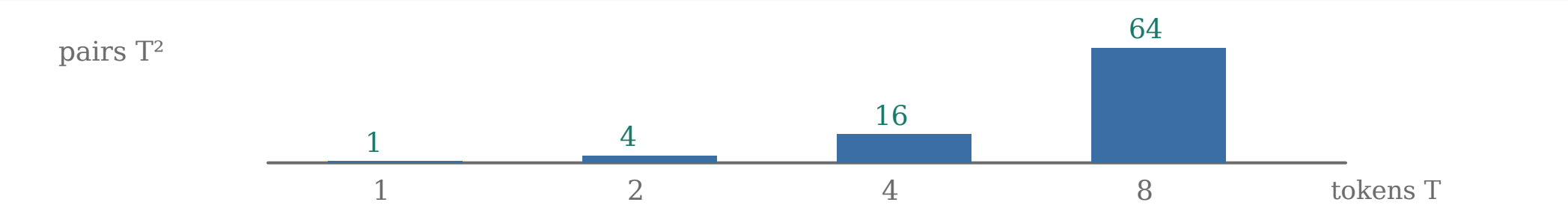
Pairwise comparison creates quadratic work

Dense self-attention, one item, equal query/key/value head widths.

$$QK^T : O(T^2 d_h), \quad PV : O(T^2 d_h), \quad Hd_h = D$$

Q, K, P, V	queries, keys, weights, values
T, D	token count, total width
H, d_h	head count, width per head

Mixing across heads costs $O(T^2 D)$. Dense projections cost $O(TD^2)$.



Query-key pair counts grow quadratically with token length.

Vaswani et al. (2017), §3.2.2

EXAMPLE

Cost and the KV cache

Memory follows the score shape

$$B = 1,\ H = 8,\ T = 1024: \quad BHT^2 = 8{,}388{,}608$$

B, H, T batch, heads, tokens

At 2 bytes per entry: 16 MiB for one materialized score tensor.

Vaswani et al. (2017), §3.2.2 · Course example

EXERCISE

Cost and the KV cache

Transfer: a longer sequence

Keep batch size B, head count H and datatype fixed. Double sequence length T. How does the memory for a materialized score tensor of shape [B,H,T,T] change?

A ×2

B ×4

C Unchanged

Vaswani et al. (2017), §3.2.2 · Course example

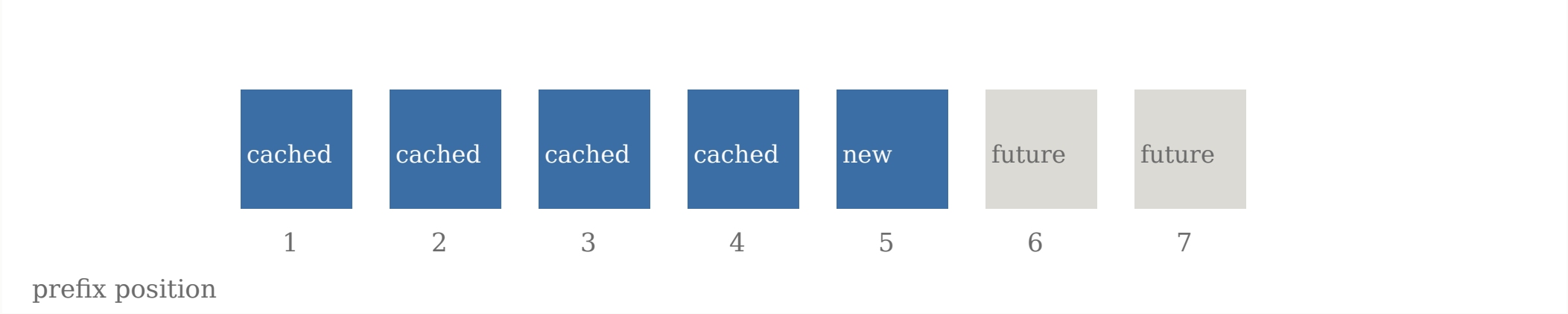
DEFINITION

Cost and the KV cache

Unchanged causal prefixes permit reuse

Autoregressive decoding adds one token at a time. A key-value (KV) cache stores earlier keys and values for reuse.

Earlier causal states cannot read appended tokens. The cache remains exact if earlier inputs, positions and parameters stay fixed.



An unchanged prefix stays cached while one new row is appended.

Shazeer (2019), §2.4–2.4.1

DERIVATION

Cost and the KV cache

One new query reads all cached keys

$$q_t^T K_{1:t}^T : [1, d_h][d_h, t] = [1, t]$$

q_t, d_h new query column and head width

$K_{1:t}, V_{1:t}$ all t key and value rows, including the appended rows

Softmax over t scores, then multiply by $V_{1:t}$. Across heads of total width D, time and cache storage are $O(tD)$ per layer and batch item.

Shazeer (2019), §2.4–2.4.1

EXERCISE

Cost and the KV cache

Transfer: can this memory be reused?

Camera tokens are replaced by a new frame. Can their old cross-attention keys and values serve as exact representations of the new frame?

A Yes, when shapes match

B No, recompute the changed memory

C Yes, if action horizon is fixed

Shazeer (2019), §2.4–2.4.1 · Course example

EXAMPLE

Cost and the KV cache

Cache validity and cumulative decoding cost

$$\sum_{t=1}^{T} t = \frac{T(T+1)}{2}, \quad \sum_{t=1}^{T} t^2 = \frac{T(T+1)(2T+1)}{6}$$

t, T current and final prefix length

D total width across heads

Cached mixing costs $O(T^2 D)$. Dense reruns score t^2 pairs before masking: $O(T^3 D)$.

Shazeer (2019), §2.4–2.4.1 · Course example

EXERCISE

Cost and the KV cache

Transfer: count incremental comparisons

Generate four tokens from an empty prefix. Count one head’s score dot products, including the current token. The dense full-prefix baseline scores every pair before applying its causal mask. Compare with cached decoding.

A Cached 10; recomputed 30

B Cached 4; recomputed 16

C Cached 16; recomputed 16

Shazeer (2019), §2.4–2.4.1 · Course example

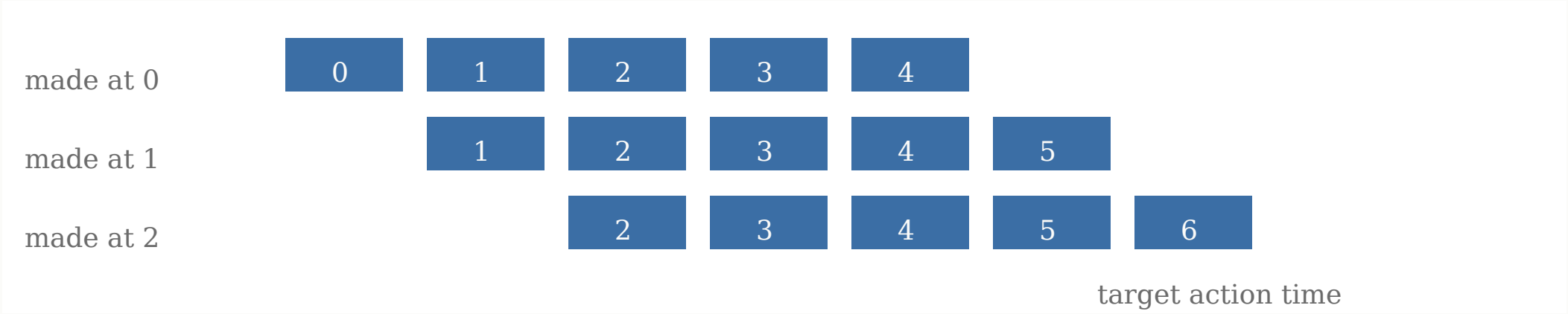
MOTIVATION

Action chunking

A coordinated movement spans several decisions

Action chunking predicts several consecutive future motor targets together from the current observation.

The robot can execute a prefix, gather new observations, and predict again.



Each row is a chunk prediction; columns are target action times.

Zhao et al. (2023), §IV-A

DEFINITION

Action chunking

Prediction time and target time are different indices

$$\widehat{A}_t \in \mathbb{R}^{K \times d_a}, \quad \widehat{A}_t[j, :] = \hat{a}^T_{t+j|t}, \quad 0 \le j < K$$

$\widehat{A}_t, t$	predicted chunk; observation and prediction time
K, d_a	chunk length, action dimension
$j, \hat{a}_{t+j\|t}$	row offset; column predicting target t+j from time t

Zhao et al. (2023), §IV-A

EXERCISE

Action chunking

Chunk length is not execution length

At prediction time t, a policy predicts $K = 6$ motor targets starting at time t. It executes the first $E = 2$ before observing again. Which target times are executed?

A Times t through t+5

B Times t and t+1

C Only time t+5

Zhao et al. (2023), §IV-A · Course example

DEFINITION

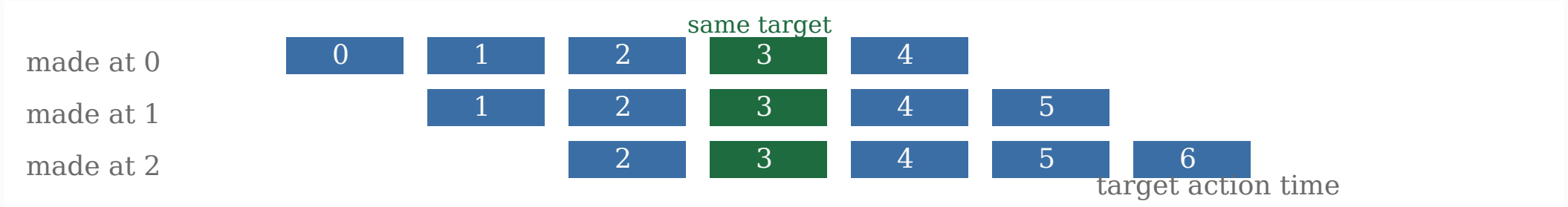
Temporal ensembling

Overlapping chunks predict the same target

Average candidates for target time s.

$$\bar{a}_s = \frac{\sum_{j=0}^{J-1} w_j \hat{a}_{s|r_j}}{\sum_{j=0}^{J-1} w_j}, \qquad w_j = e^{-mj}$$

- j, r_j oldest-first index; chunk prediction time
- $J, \hat{a}_{s|r_j}, \bar{a}_s$ chunk count; candidate; ensemble action
- $w_j, m \geq 0$ weight and decay rate



Combine one target-time column across overlapping chunks.

Zhao et al. (2023), §IV-A

EXAMPLE

Temporal ensembling

One target, two predictions

$$\hat{a}_{s|r_0} = 2, \quad \hat{a}_{s|r_1} = 4, \quad w_j = e^{-mj}, \quad m = \log 2$$

s	common target time
r_0, r_1	oldest and newest prediction times
$\hat{a}_{s\|r_j}, \bar{a}_s$	candidate from chunk j; ensemble action

$w = (1, 1/2)$, so $\bar{a}_s = (2 + 2)/(3/2) = 8/3$.

Zhao et al. (2023), §IV-A · Course example

EXERCISE

Temporal ensembling

Transfer: three estimates for one target

Oldest-to-newest predictions for target s are 1, 3, 5. Use $w_j = e^{-mj}$ for $j = 0, 1, 2$, with $m = \log 2$. Find the normalized weighted average.

A 3

B 15/7

C 27/7

Zhao et al. (2023), §IV-A · Course example

INTERPRETATION

Temporal ensembling

Averages need compatible action representations

Average predictions for the same target time in the same coordinate system.

Smoothing alone does not enforce collision, dynamics or joint-limit constraints.

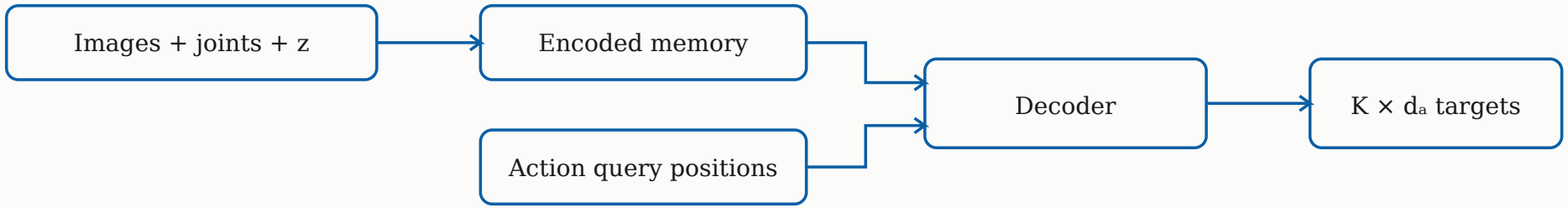
Zhao et al. (2023), §IV-A

DEFINITION

ACT: observations to actions

ACT combines observation tokens with action queries

Action Chunking with Transformers (ACT) predicts K motor targets of width d_a from observation tokens. A latent variable z is an unobserved input representing variation among demonstrations.



Zhao et al. (2023), §IV-C and Algorithm 1

EXAMPLE

ACT: observations to actions

A concrete observation-token count

$$4(15 \cdot 20) + 2 = 1202$$

4	camera views in the ACT paper
15×20	feature grid per view
$+2$	joint-state and latent tokens

Each token has 512 features in the reported model. Spatial positions accompany image features.

Zhao et al. (2023), §IV-C and Algorithm 1 · Course example

DEFINITION

ACT: observations to actions

Action queries use cross-attention

$$S_a = \frac{Q_a K_o^T}{\sqrt{d_k}} \in \mathbb{R}^{K \times T_o}$$

S_a	action-by-observation raw scores
Q_a, K_o	action query rows; observation key rows
K, T_o, d_k	action slots; observation tokens; shared feature width

Softmax of each score row gives the weights used to mix observation values.

Zhao et al. (2023), §IV-C and Algorithm 1

EXERCISE

ACT: observations to actions

Transfer: encoder tokens and action slots

A model has 20 observation tokens and predicts a chunk of 6 actions. Each action query reads every observation token. What is one head’s cross-attention score shape?

A. 20×20

B. 6×20

C. 6×6

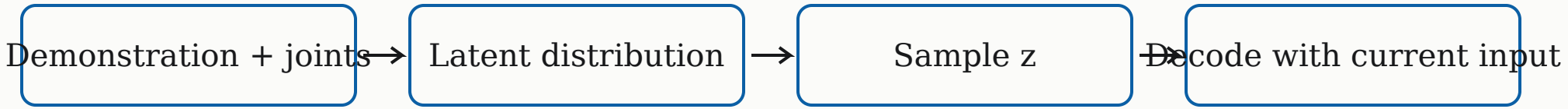
Zhao et al. (2023), §IV-C and Algorithm 1 · Course example

MOTIVATION

ACT: latent-variable training

Demonstrations can differ at the same observation

A latent variable z is an unobserved input that represents variation among demonstrated chunks. A conditional variational autoencoder (CVAE) learns an encoder distribution for z and a decoder conditioned on observations.



Zhao et al. (2023), §IV-B–C, Appendix C

DEFINITION

ACT: latent-variable training

The encoder predicts a diagonal Gaussian

Diagonal Gaussian covariance gives independent coordinates.

$$q_\phi(z \mid A_t, r_t) = \mathcal{N}(\mu, \mathrm{diag}(\sigma^2)), \quad p(z) = \mathcal{N}(0, I_L)$$

z, μ, σ	latent; means; positive scales
A_t, r_t	demonstrated chunk; observed joints at time t
q_ϕ, p, I_L	encoder; prior; identity for L latent coordinates

Kingma & Welling (2014), §2, Appendix B

DERIVATION

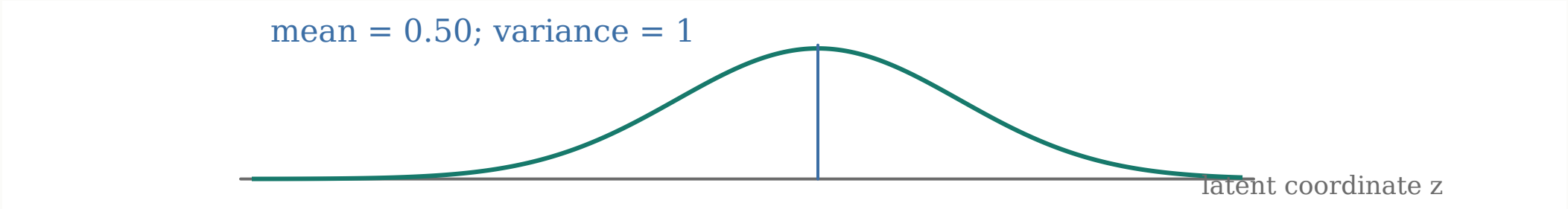
ACT: latent-variable training

Reparameterization exposes a differentiable path

Reparameterization separates noise from learned parameters.

$$\epsilon \sim \mathcal{N}(0, I_L), \quad z = \mu + \sigma \odot \epsilon$$

- ϵ, I_L Gaussian noise; latent identity of size L
- μ, σ means and positive scales; ⊙ multiplies elementwise
- z latent sample



Changing the mean shifts a Gaussian without changing its width.

Kingma & Welling (2014), §2, Appendix B

EXERCISE

ACT: latent-variable training

Log-variance is not the standard deviation

For one latent coordinate, mean $\mu = 1$, log-variance $\ell = \log \sigma^2 = \log 4$, and noise $\epsilon = -1$. Find the sample $z = \mu + \sigma\epsilon$ using standard deviation σ.

A -3

B -1

C 1

Kingma & Welling (2014), §2, Appendix B · Course example

DERIVATION

ACT: latent-variable training

A bound makes the latent objective tractable

$$p(A, z|o) = p(A|o)P(z)$$

$$\mathbb{E}_q \log[p(A, z|o)/q(z)] = \log p(A|o) - D_{KL}(q\|P)$$

A, o, z demonstrated chunk; fixed observation; latent

q, P encoder; true posterior given A and o

D_{KL} Kullback–Leibler divergence; average log ratio under q

Assume positive finite evidence and finite posterior KL.

Kingma & Welling (2014), §2, Appendix B

DERIVATION

ACT: latent-variable training

Nonnegative KL gives the evidence lower bound

$$\begin{aligned}\mathcal{L} &= \mathbb{E}_q \log[p(A|o,z)p(z)/q(z)] \\ &= \mathbb{E}_q \log p(A|o,z) - D_{KL}(q\|p) \le \log p(A|o)\end{aligned}$$

$\mathcal{L}$	evidence lower bound (ELBO)
A, o, z	chunk, observation, latent
q, p	encoder, prior
$p(A\|o,z), p(A\|o)$	likelihood, evidence

Posterior KL is the nonnegative gap.

Kingma & Welling (2014), §2, Appendix B

EXERCISE

ACT: latent-variable training

Transfer: which divergence creates the gap?

For chunk A, observation o and latent z, q is the encoder, $p_\theta(z \mid A, o)$ the true posterior and $p(z)$ the prior. Which gap G makes $\log p_\theta(A \mid o) = \mathcal{L}_{\text{ELBO}} + G$ exact? ELBO means evidence lower bound.

A $D_{KL}(q \| p_\theta(z \mid A, o))$

B $-D_{KL}(q \| p(z))$

C The action prediction error alone

Kingma & Welling (2014), §2, Appendix B · Course example

DEFINITION

ACT: latent-variable training

ACT balances reconstruction and latent regularization

$$\mathcal{L} = \frac{1}{K d_a} \sum_{j=0}^{K-1} \sum_{r=1}^{d_a} |\hat{A}_{jr} - A_{jr}| + \beta D_{KL}(q_\phi \| \mathcal{N}(0, I))$$

$A, \hat{A}$	target/predicted chunks; all rows valid
K, d_a	chunk length; action width
j, r	row, feature
$\mathcal{L}, \beta \geq 0$	loss, KL weight
$q_\phi, \mathcal{N}(0, I)$	encoder; standard Gaussian prior

Zhao et al. (2023), §IV-C and Algorithm 1

DERIVATION

ACT: latent-variable training

Gaussian KL follows from a log-density ratio

$$\log \frac{q_j(z_j)}{p_j(z_j)} = -\log \sigma_j - \frac{(z_j - \mu_j)^2}{2\sigma_j^2} + \frac{z_j^2}{2}$$

q_j, p_j	encoder $N(\mu_j,\sigma_j^2)$, prior $N(0,1)$
z_j, μ_j, σ_j	sample, mean, positive scale
$\mathbb{E}_q$	average under encoder

$$\mathbb{E}_q[(z_j - \mu_j)^2] = \sigma_j^2;\ \mathbb{E}_q[z_j^2] = \mu_j^2 + \sigma_j^2.$$

Kingma & Welling (2014), §2, Appendix B

DERIVATION

ACT: latent-variable training

The diagonal-Gaussian KL is explicit

$$D_{KL}(q\|p) = \sum_{j=1}^{L} \mathbb{E}_q \log(q_j/p_j)$$

$$= \tfrac{1}{2} \sum_{j=1}^{L} (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$$

q, p	diagonal encoder Gaussian; standard Gaussian prior
$q_j, p_j, \mu_j, \sigma_j$	coordinate densities; encoder mean; positive scale
j, L	latent index and width

Independence makes the log ratios add.

Kingma & Welling (2014), §2, Appendix B

EXAMPLE

ACT: latent-variable training

A latent penalty calculated by hand

$$\mu = (1, 0)^T, \quad \sigma^2 = (1, 4)^T$$

μ, σ^2	coordinate means and variances of a diagonal Gaussian encoder
$\mathcal{N}(0, I_2)$	standard two-dimensional Gaussian prior

KL to the prior: $D_{KL} = \frac{1}{2}[1 + (4 - 1 - \log 4)] = 2 - \log 2 \approx 1.3069.$

Kingma & Welling (2014), §2, Appendix B · Course example

EXERCISE

ACT: latent-variable training

Transfer: combine the two loss terms

Four action errors are $1, -1, 0, 2$. Gaussian encoder mean $\mu = 1$, variance $\sigma^2 = 1$. Find the mean absolute error plus $\beta = 2$ times the KL from the encoder to the standard normal prior $\mathcal{N}(0, 1)$.

A 1

B 2

C 5

Zhao et al. (2023), §IV-B–C, Appendix C · Course example

DEFINITION

ACT: deployment

The deployed ACT policy fixes the latent to zero

$$\hat{A}_t = f_\theta(o_t, z = 0)$$

$\hat{A}_t, o_t$	predicted action chunk; images and joints at time t
f_θ	trained observation encoder and action decoder
$z = 0$	latent fixed at the standard Gaussian prior mean

The demonstration-conditioned latent encoder is used during training.

Zhao et al. (2023), §IV-B–C, Appendix C

EXERCISE

ACT: deployment

Training and inference use different information

The latent input is z. Which input route matches the reported ACT inference procedure?

A. Encode the unknown future target chunk

B. Current observations plus z=0

C. Draw z from the demonstration posterior

Zhao et al. (2023), §IV-B–C, Appendix C · Course example

SYNTHESIS

ACT: deployment

One target from observation to execution

Query 1, keys $(0, \log 3)$, values $(0, 4)$, identity readout. Older candidate: 1 for target s.

$$(0, \log 3) \xrightarrow{\text{softmax}} (1/4, 3/4) \longrightarrow \hat{a}_{s|s} = 3$$

$\hat{a}_{s|s}$ new candidate at s

$\bar{a}_s$ executed ensemble target

Oldest-first weights $(1, 1/2)$ give $\bar{a}_s = (1 + (1/2)3)/(1 + 1/2) = 5/3$.

Observe and replan with latent zero.

Zhao et al. (2023), §IV-B–C, Appendix C

EXERCISE

ACT: deployment

Transfer: trace the live policy loop

At target time s, two chunks predict actions 2 and 5, oldest first. Use $w_j = e^{-mj}$ with $m = \log 2$ and $j = 0, 1$. A fresh image arrives after execution. Find the executed target and next computation.

A. Execute 3; encode the fresh observation

B. Execute 7/2; change the KL weight

C. Execute 5; reuse all old image features

Zhao et al. (2023), §IV-B–C, Appendix C · Course example

EXERCISE

ACT: deployment

Transfer: a changed scene after prediction

A chunk predicts six actions. After one executed action, a fresh observation shows that the object moved. Which response uses the new observation?

A. Keep executing all old targets without recomputing

B. Encode the new observation and predict a new chunk

C. Change the KL coefficient at inference

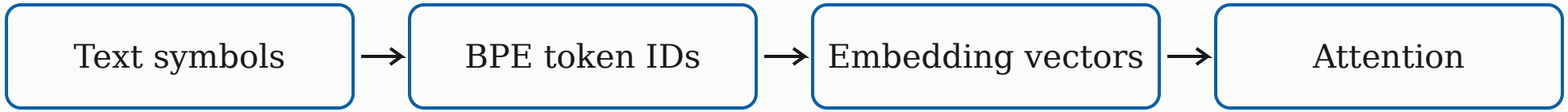
Zhao et al. (2023), §IV-B–C, Appendix C · Course example

BRIDGE

Discrete tokens and continuous actions

Byte-pair encoding is a text representation choice

Byte-pair encoding (BPE) builds a subword vocabulary by repeatedly merging frequent adjacent symbol pairs. A tokenizer maps text to discrete token IDs.



Sennrich, Haddow & Birch (2016), §3.2

DEFINITION

Discrete tokens and continuous actions

Top-p restricts a categorical sampling set

A categorical distribution gives probabilities to discrete choices.

$$\sum_{j=1}^{k-1} p_{(j)} < \rho \leq \sum_{j=1}^{k} p_{(j)}$$

$p_{(1)} \geq p_{(2)} \geq \cdots$	sorted category probabilities
$\rho \in (0, 1]$	cumulative mass threshold
k	smallest retained prefix size

Renormalize over the retained categories, then sample one.

Holtzman et al. (2020), §3.1

EXERCISE

Discrete tokens and continuous actions

Transfer: the retained set depends on mass

Sorted probabilities are $(0.5, 0.3, 0.2)$. Keep the smallest prefix whose sum reaches threshold $\rho = 0.7$, then renormalize. Which distribution is sampled?

A $(0.5, 0.3, 0.2)$

B $(0.625, 0.375, 0)$

C $(1, 0, 0)$

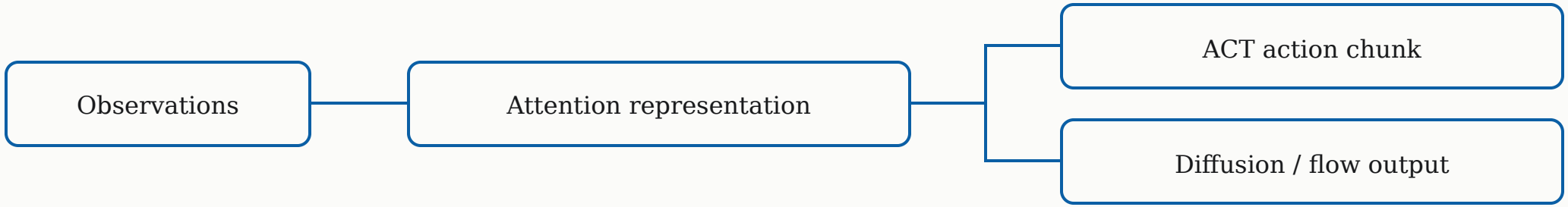
Holtzman et al. (2020), §3.1 · Course example

SYNTHESIS

Transformers and ACT

One representation, several output models

Attention builds a conditional representation. ACT maps it to an action chunk. Diffusion and flow matching will use conditioning to guide iterative generation.



Zhao et al. (2023), §IV-B–C, Appendix C

References

Primary sources and reading 1

1. Vaswani et al. (2017). Attention Is All You Need. §3.2–3.5, Eq. 1; NeurIPS.
2. Blanchard, Higham & Higham (2021). Accurately Computing the Log-Sum-Exp and Softmax Functions. §4.

References

Primary sources and reading 2

1. Su et al. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. §3.2.
2. Xiong et al. (2020). On Layer Normalization in the Transformer Architecture. §3.1, Table 1; ICML.
3. Shazeer (2019). Fast Transformer Decoding: One Write-Head is All You Need. §2.4–2.4.1.

References

Primary sources and reading 3

1. Zhao et al. (2023). Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware. §IV and Appendix C; RSS.
2. Kingma & Welling (2014). Auto-Encoding Variational Bayes. §2, Appendix B; ICLR.

References

Primary sources and reading 4

1. Sennrich, Haddow & Birch (2016). Neural Machine Translation of Rare Words with Subword Units. §3.2; ACL.
2. Holtzman et al. (2020). The Curious Case of Neural Text Degeneration. §3.1; ICLR.